PGCP-AI STUDENT – PROPOSED TECHNICAL CONTRIBUTION

Role: AI Engineer – Code Understanding & Intelligent Review Engine

The PGCP-AI student will be responsible for designing and developing the **AI and code-understanding component** of the proposed system.

The objective of this contribution is to develop an intelligent code-review engine that can analyze source code, identify potential issues, explain the findings, assign appropriate severity and generate possible code improvements.

The AI component will be developed as an independent service that communicates with the main application backend through APIs.

The following components are proposed as the primary responsibilities of the PGCP-AI student.

1. Programming Language Detection

Develop a mechanism to identify or validate the programming language of submitted source code.

The proposed approach may consider:

- File-extension based detection
- Content-based detection
- Parser-based validation
- Manual language selection as a fallback

The final approach and supported programming languages will be finalized based on research and project feasibility.

2. Code Parsing and Code Representation

Develop the code-understanding layer responsible for analyzing the structure of submitted source code.

The proposed work includes:

- Abstract Syntax Tree (AST) generation
- Code structure extraction
- Function and class identification
- Syntax and structural analysis
- Parser integration
- Code representation suitable for AI analysis

Possible technologies such as **Tree-sitter or language-specific parsers** will be evaluated before final implementation.

The purpose of this module is to provide structured information about the source code instead of depending only on raw text.

3. AI-Based Code Review Engine

Develop the core AI review pipeline responsible for analyzing source code and generating meaningful review findings.

Depending on the selected scope and supported languages, the system may analyze:

- Potential bugs
- Logic issues
- Code-quality problems
- Bad programming practices
- Maintainability issues
- Potential security-related issues
- Performance-related issues

The exact review categories will be finalized after evaluating their feasibility and reliability.

The AI engine will focus on providing **context-aware analysis and explanations**, rather than simply generating generic coding advice.

4. LLM / Code Model Integration

Research, evaluate and integrate an appropriate **code-capable AI/LLM model** for code understanding and review.

The evaluation will consider:

- Code-understanding capability
- Bug-detection capability
- Refactoring capability
- Context-window requirements
- Structured-output support
- Latency
- Cost
- Privacy
- Programming-language support
- Hosted versus local deployment

Both hosted and locally deployable models may be evaluated where appropriate.

The final model will be selected based on project requirements, performance and available development resources.

5. Hybrid Static Analysis + AI Architecture

The proposed AI architecture will investigate combining deterministic code analysis with AI reasoning.

Conceptually:

Source Code

↓

Code Parsing / AST

↓

Static Analysis + AI/LLM Analysis

↓

Review Aggregator

↓

Structured Findings

The static-analysis component can provide deterministic findings, while the AI component can provide contextual reasoning, explanations and improvement suggestions.

This approach is intended to reduce dependence on free-form LLM responses and improve the reliability of the review system.

The project specification specifically identifies the combination of static analysis, code parsing and LLM reasoning as an area to investigate.

6. Severity and Confidence Classification

Develop the logic for assigning severity to identified findings.

The proposed severity levels are:

- Critical
- High
- Medium
- Low
- Informational

The final classification methodology will be determined through research and evaluation.

The system will also consider **confidence separately from severity**.

Severity: High
Confidence: 0.91

This distinction will help differentiate the potential impact of an issue from the confidence of the AI system in its finding.

The project specification explicitly identifies severity classification and the distinction between severity and confidence as areas for investigation.

7. Structured AI Output

The AI service will produce structured review results instead of returning only free-form text.

A review finding may contain information such as:

- Finding location
- Line number/range

- Issue category
- Severity
- Confidence
- Problem description
- Explanation
- Suggested solution
- Suggested replacement code

The structured result will then be passed to the PGCP-AC backend for storage and presentation through the web application.

The project specification identifies structured JSON output, schema validation and reliable AI-output handling as important parts of the AI component.

8. AI-Based Refactoring Suggestions

Develop the AI capability to suggest improvements to problematic or inefficient code.

Original Code

↓

Identified Problem

↓

Suggested Improvement

↓

Modified Code / Patch

The feasibility of automatically validating generated changes will also be investigated.

Where technically practical, a proposed workflow may include:

AI Suggestion → Patch Generation → Validation → Final Suggestion

The validation stage may involve static analysis and, where safely possible, compilation or testing in an isolated environment.

Generated code will not be treated as automatically correct without validation.

The project specification identifies patch generation and validation as areas for research.

Overall PGCP-AI Deliverable

The final PGCP-AI contribution will be an **AI Code Review Engine** capable of accepting structured source-code input and producing structured review results.

Proposed AI Pipeline

Source Code

↓

Language Detection

↓

Code Parsing / AST

↓

Code Representation

↓

Static Analysis

–

AI/LLM Analysis

↓

Finding Aggregation

↓

Severity + Confidence

↓

Explanation

↓

Refactoring Suggestion

↓

Structured Review JSON

↓

Spring Boot Backend

↓

Frontend Review Dashboard

Why PGCP-AI is Required in the Project

The AI contribution is an independent technical component of the project and cannot be replaced by simply connecting the application to an external LLM API.

The PGCP-AI student will be responsible for:

- Code understanding
- Code parsing
- AI/LLM evaluation
- AI review pipeline
- Bug detection
- AI reasoning
- Severity classification

- Structured AI output
- Refactoring suggestions
- AI evaluation

The PGCP-AC students will be responsible for the surrounding software-engineering infrastructure, including the frontend, Spring Boot backend, database, APIs, testing, security, deployment and DevOps.

Therefore, the proposed division creates a meaningful interdisciplinary contribution from both **PGCP-AI and PGCP-AC**.

Proposed Scope at the Approval Stage

The above components represent the **proposed PGCP-AI scope for project approval**.

The exact:

- Programming languages
- AI model
- Parser
- Static-analysis tools
- Evaluation dataset
- Severity methodology
- Refactoring-validation approach

will be finalized during the requirement-analysis and research phase after project approval.

This approach allows the team to select technically feasible methods while maintaining the proposed AI contribution and interdisciplinary project objectives.